LAPORAN WORKSHOP DEVELOPER OPERASIONAL
MINGGU KE – 9

Disusun Oleh :

Chiquita Clairina Kyrevi

E41230019

Golongan A

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

2026

DAFTAR ISI

DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB 1 HASIL DAN PEMBAHASAN 1
1.1 Mengenal dan konfigurasi Docker Swarm (Acara 17) 1
1.2 Mengelola Service pada Docker Swarm (Acara 18) 4

DAFTAR GAMBAR

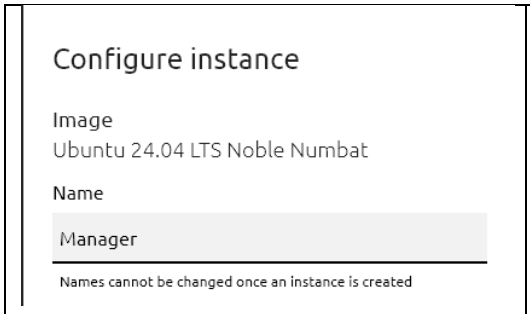
Gambar 1. 1 VM untuk Manager 1
Gambar 1. 2 VM untuk Worker Node 1
Gambar 1. 3 Ip masing masing VM 1
Gambar 1. 4 Pengecekan Ip dan ping 2
Gambar 1. 5 Pengecekan firewall 2
Gambar 1. 6 Instalasi docker 3
Gambar 1. 7 Inisialisi Docker Swarm di Manager 3
Gambar 1. 8 Join worker ke swarm manager 3
Gambar 1. 9 Cek status swarm di manager 4
Gambar 1. 10 Cek Node yang terhubung 4
Gambar 1. 11 Pengecekan dan Pembuatan Service 4
Gambar 1. 12 Pengecekan Service di web 5
Gambar 1. 13 Update Service 5
Gambar 1. 14 Pengecekan kembali hasil update 5
Gambar 1. 15 Inspect service 6
Gambar 1. 16 Cek Service 6
Gambar 1. 17 Keluar swarn untuk worker node 7
Gambar 1. 18 Keluar swarm untuk manager 7

BAB 1
HASIL DAN PEMBAHASAN

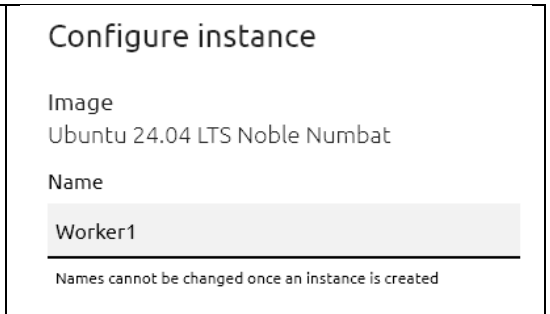
1.1 Mengenal dan konfigurasi Docker Swarm (Acara 17)

1. Membuat 2 Virtual Private Server (VPS) menggunakan Multipass

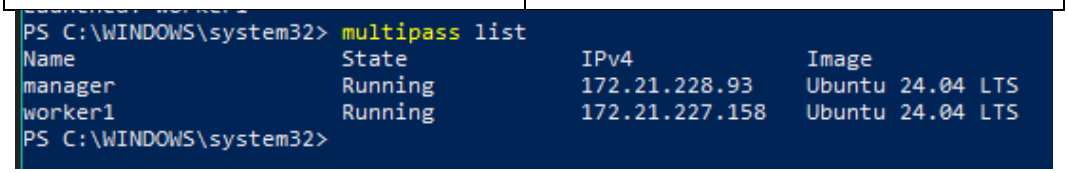
Pada tahap ini dilakukan pembuatan dua buah instance virtual menggunakan Multipass yang akan berperan sebagai node dalam arsitektur Docker Swarm. Satu instance akan dikonfigurasikan sebagai manager node yang bertugas mengatur cluster, sedangkan instance lainnya akan berperan sebagai worker node yang menjalankan service yang didistribusikan oleh manager



Gambar 1. 1 VM untuk Manager



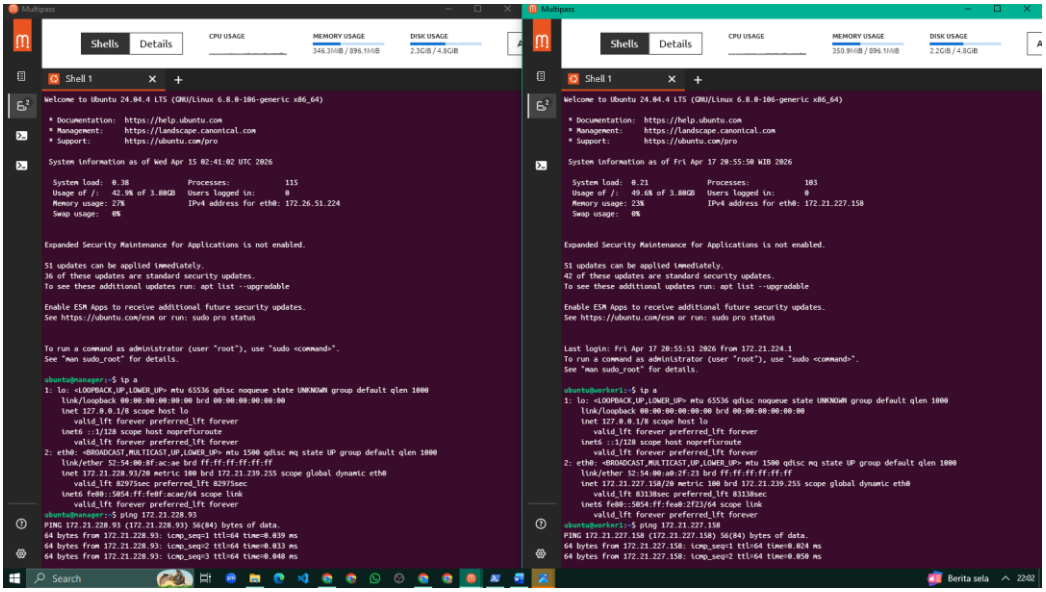
Gambar 1. 2 VM untuk Worker Node



Gambar 1. 3 Ip masing masing VM

2. Melakukan pengecekan IP Address dan konektivitas antar VPS

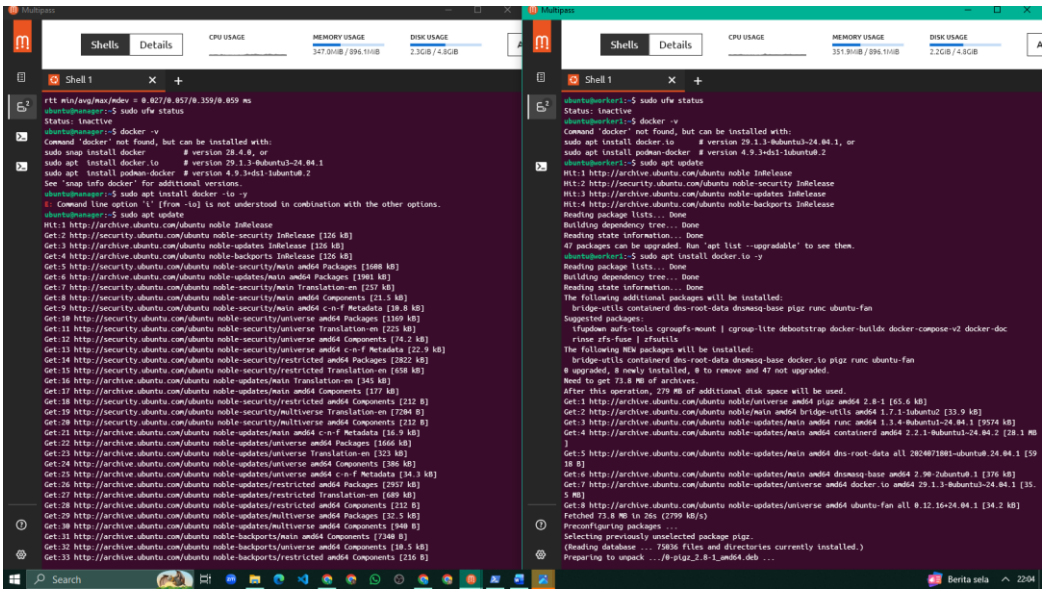
Setelah kedua VPS berhasil dibuat, langkah selanjutnya adalah memastikan bahwa masing-masing instance memiliki alamat IP serta dapat saling terhubung. Hal ini penting karena komunikasi antar node dalam Docker Swarm bergantung pada konektivitas jaringan yang stabil. Pengujian koneksi biasanya dilakukan dengan perintah ping dari satu node ke node lainnya.



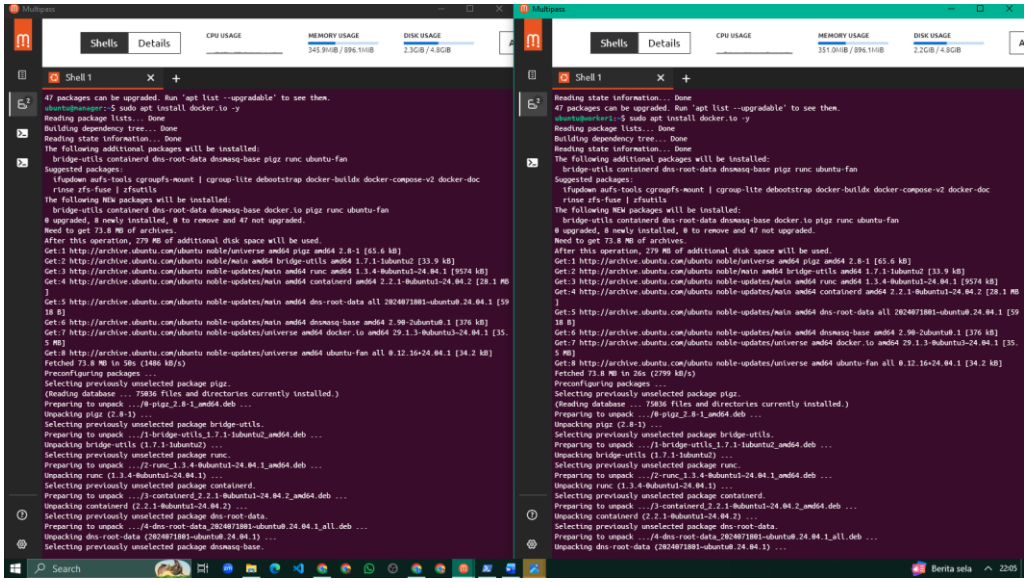
Gambar 1. 4 Pengecekan Ip dan ping

3. Melakukan pengecekan firewall dan instalasi Docker pada kedua VPS

Tahap ini bertujuan untuk memastikan bahwa tidak ada aturan firewall yang menghambat komunikasi antar node. Jika firewall aktif, maka perlu dilakukan pembukaan port yang digunakan oleh Docker Swarm. Selain itu, dilakukan instalasi Docker sebagai platform utama untuk menjalankan container dan mengelola service dalam cluster.



Gambar 1. 5 Pengecekan firewall



Gambar 1. 6 Instalasi docker

4. Melakukan inisialisasi Docker Swarm pada node Manager

Pada tahap ini, node manager diinisialisasi menggunakan perintah Docker Swarm. Proses ini akan mengubah node menjadi pusat kontrol cluster dan menghasilkan token yang nantinya digunakan oleh node lain untuk bergabung ke dalam swarm. Manager bertanggung jawab dalam pengaturan scheduling, deployment, dan monitoring service.

```
ubuntu@manager:~$ sudo docker swarm init --advertise-addr 172.21.228.93
Swarm initialized: current node (7wjlei7pupwxfi3w8h0kwiem9) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0z04r3lha6qz02v54hkrsihxdlu432feckxoutjt256fgk8ufj-7xyk536k6zjvhd797evsv7
nw7 172.21.228.93:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

Gambar 1. 7 Inisialisi Docker Swarm di Manager

5. Menghubungkan node Worker ke dalam cluster Swarm

Setelah manager berhasil diinisialisasi, langkah berikutnya adalah menggabungkan worker ke dalam cluster menggunakan token yang telah dihasilkan sebelumnya. Dengan bergabungnya worker, maka beban kerja dapat didistribusikan sehingga meningkatkan ketersediaan dan skalabilitas layanan.

```
ubuntu@worker1:~$ sudo docker swarm join --token SWMTKN-1-0z04r3lha6qz02v54hkrsihxdlu432feckxoutjt256fgk8ufj-7xy
k536k6zjvhd797evsv7nw7 172.21.228.93:2377
This node joined a swarm as a worker.
```

Gambar 1. 8 Join worker ke swarm manager

6. Melakukan pengecekan status Docker Swarm pada Manager

Tahap ini dilakukan untuk memastikan bahwa Docker Swarm telah aktif dan berjalan dengan baik pada node manager. Informasi yang ditampilkan biasanya mencakup status swarm, jumlah node, serta peran dari masing-masing node dalam cluster.

```
ubuntu@manager:~$ sudo docker info | grep -i swarm
 Swarm: active
```

Gambar 1. 9 Cek status swarm di manager

7. Melihat daftar node yang terhubung dalam cluster

Pada langkah ini dilakukan pengecekan terhadap seluruh node yang telah terhubung ke dalam swarm. Hal ini bertujuan untuk memastikan bahwa proses join worker berhasil dan node telah dikenali oleh manager.

```
ubuntu@manager:~$ sudo docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
7wjlei7pupwxfi3w8h0kwiem9 *   manager    Ready     Active         Leader           29.1.3
hnht9nlml8oqmg68q1zcgbx83     worker1    Ready     Active                          29.1.3
```

Gambar 1. 10 Cek Node yang terhubung

1.2 Mengelola Service pada Docker Swarm (Acara 18)

1. Membuat service pada Docker Swarm & Melakukan pengecekan service yang telah dibuat

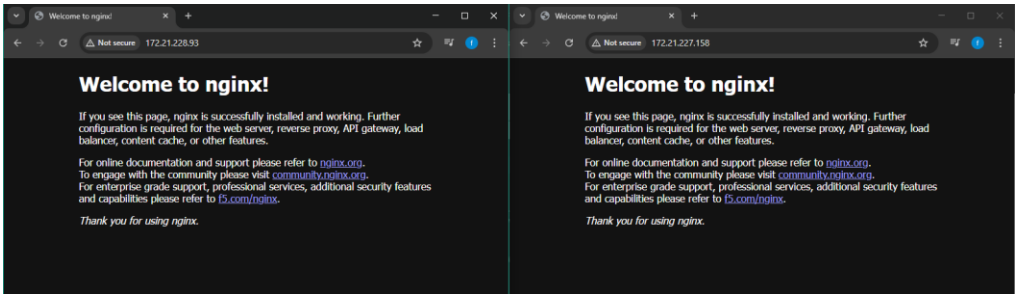
Service merupakan komponen utama dalam Docker Swarm yang digunakan untuk menjalankan aplikasi dalam bentuk container. Pada tahap ini dibuat sebuah service sederhana menggunakan image Nginx yang nantinya dapat diakses melalui browser. Setelah service berhasil dibuat, dilakukan pengecekan untuk memastikan bahwa service berjalan dengan baik. Informasi yang ditampilkan meliputi nama service, jumlah replika, dan statusnya.

```
ubuntu@manager:~$ sudo docker service create --name web -p 80:80 nginx
z6ne56kggea2ji1j47uzfouic
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service z6ne56kggea2ji1j47uzfouic converged
ubuntu@manager:~$ docker service ls
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@manager:~$ sudo docker service ls
ID             NAME      MODE         REPLICAS   IMAGE          PORTS
z6ne56kggea2   web       replicated   1/1        nginx:latest   *:80->80/tcp
```

Gambar 1. 11 Pengecekan dan Pembuatan Service

2. Mengakses service melalui browser

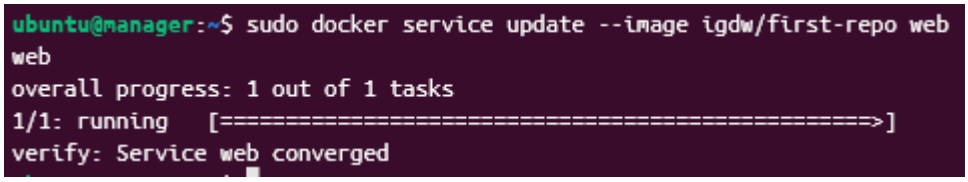
Service yang telah dibuat kemudian diuji dengan mengaksesnya melalui browser menggunakan alamat IP node dan port yang telah ditentukan. Docker Swarm menggunakan sistem routing mesh sehingga service dapat diakses dari node mana saja dalam cluster.



Gambar 1. 12 Pengecekan Service di web

3. Melakukan update terhadap service

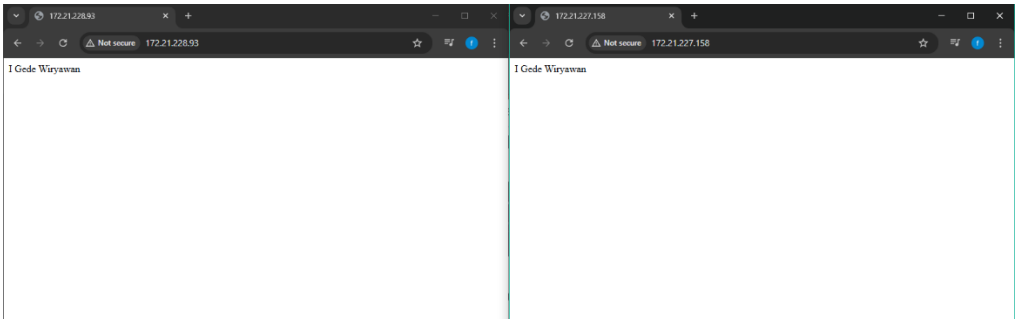
Pada tahap ini dilakukan perubahan konfigurasi service, seperti menambah jumlah replika. Hal ini bertujuan untuk menguji kemampuan scaling dari Docker Swarm dalam menangani peningkatan beban kerja.



Gambar 1. 13 Update Service

4. Melakukan pengecekan hasil update service

Setelah update dilakukan, perlu dilakukan verifikasi untuk memastikan bahwa perubahan telah diterapkan dengan benar dan container tambahan telah berjalan sesuai dengan konfigurasi baru.



Gambar 1. 14 Pengecekan kembali hasil update

5. Melakukan inspect terhadap service di Manager

Perintah inspect digunakan untuk melihat detail konfigurasi service secara lengkap, termasuk pengaturan jaringan, jumlah replika, serta parameter lainnya.

```
ubuntu@manager:~$ sudo docker service inspect web --pretty

ID:             z6ne56kggea2ji1j47uzfouic
Name:           web
Service Mode:   Replicated
 Replicas:      1
UpdateStatus:
 State:         completed
 Started:       4 minutes ago
 Completed:     3 minutes ago
 Message:       update completed
Placement:
UpdateConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Update order:      stop-first
RollbackConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Rollback order:    stop-first
ContainerSpec:
 Image:         igdw/first-repo:latest
 Init:          false
Resources:
Endpoint Mode:  vip
Ports:
 PublishedPort = 80
  Protocol = tcp
  TargetPort = 80
  PublishMode = ingress
```

Gambar 1. 15 Inspect service

6. Melihat container yang berjalan pada node Manager

Tahap ini bertujuan untuk mengetahui container apa saja yang sedang berjalan pada node manager sebagai bagian dari service yang telah dibuat.

```
ubuntu@manager:~$ sudo docker service ps web | grep Running
5gk9d482ijbt   web.1       igdw/first-repo:latest   manager   Running        Running 5 minutes ago
```

Gambar 1. 16 Cek Service

7. Worker keluar dari Swarm

Node worker dapat keluar dari cluster dengan perintah leave. Setelah keluar, node tidak lagi menjadi bagian dari swarm.

```
ubuntu@worker1:~$ sudo docker swarm leave
Node left the swarm.
```

Gambar 1. 17 Keluar swarn untuk worker node

8. Manager keluar dari Swarm

Node manager juga dapat keluar dari swarm, namun membutuhkan opsi tambahan karena perannya sebagai pengendali cluster. Setelah keluar, cluster akan berhenti jika tidak ada manager lain.

```
ubuntu@manager:~$ sudo docker swarm leave --force
Node left the swarm.
```

Gambar 1. 18 Keluar swarm untuk manager